VIVA PREPARATION

79 Viva Questions, With Answers

Every topic in the project, from the physics of the machine to the hyperparameter search — phrased the way an examiner would ask it, and answered the way you should answer it.

Project PMSM Thermal Virtual Sensor — estimating stator winding temperature from drive-measurable signals

Course UCS321 — AI for Engineers · MST Mini Project · Problem Statement 6 (GE Power Systems)

Student Lohhith Thakur · Roll No. 1025220020 · B.E. Electronics (VLSI Design & Technology)

1.54 °C	0.9965	104	3.8×
RMSE on unseen sessions	R^2 on unseen sessions	features from 8 channels	error hidden by a random split

How to use this

- Each answer is written to be *spoken*, not read out — two to six sentences, with the number that settles the point.
- **Bold** marks the figure or phrase to land. If you remember nothing else from an answer, remember the bold part.
- Sections A–J follow the notebook's own order, so you can revise a section and the corresponding notebook section together.
- If you are short of time, the six that decide the viva are **23** (how you identified leakage), **30** (deriving the EWMA from the thermal ODE), **41** (quantifying the leakage), **49** (proving extrapolation), **61** (defending $R^2 = 0.9965$) and **66** (the CV-to-test gap).

Contents

A	Problem Understanding and Objective	Q1–8
B	Dataset	Q9–16
C	Exploratory Data Analysis	Q17–20
D	Pre-processing and Target Leakage	Q21–26
E	Feature Engineering	Q27–38
F	Train / Test Split Protocol	Q39–44
G	Model Development	Q45–53
H	Performance Evaluation and Interpretation	Q54–62
I	Hyperparameter Tuning and Cross-Validation	Q63–69
J	Deployment, Limitations and Extensions	Q70–79

SECTION A — 8 QUESTIONS

Problem Understanding and Objective

1 State the problem your project solves, in one sentence.

> Estimate the stator winding temperature of a permanent-magnet synchronous motor (PMSM) from signals the drive already measures — ambient and coolant temperature, dq-frame voltages and currents, rotor speed and torque — accurately enough that a controller can use the estimate as a thermal limit instead of a physical thermocouple.

2 Why can you not simply put a thermocouple on the winding?

> Three reasons, and cost is only one. The winding is varnished, rotating-adjacent and at high voltage, so bonding a sensor to it means extra wiring routed out through a sealed housing. That wiring is an additional failure mode in the part of the machine you least want to compromise. And the bill-of-materials cost does not survive volume production. Test benches have these sensors; shipped motors do not. That gap is exactly the gap this project fills.

3 What is a virtual sensor, or soft sensor?

> A software component that outputs a quantity you cannot or will not measure directly, computed from quantities you already measure. The ‘sensor’ is a model, not hardware. Here the model replaces a winding thermocouple, using the eight channels a field-oriented-control drive already acquires for free.

4 Which temperature are you predicting, and why that one?

> **stator_winding**. It is the binding thermal constraint on the machine: the winding insulation is the first thing to fail from overheating, and insulation life halves roughly every 10 °C of sustained overtemperature. The magnet and iron temperatures matter, but the winding is what sets how hard you are allowed to push the motor.

5 What is Class-H insulation, and why does 180 °C matter?

> Class H is an insulation thermal class rated to about 180 °C continuous. Past that the varnish degrades and the failure mode is a turn-to-turn short. So 180 °C is the number the protection function is defending, and every design decision in this project — including which error metric I report — traces back to defending it.

6 What is the engineering benefit if the estimate is accurate?

> Torque and power density. A drive that does not know the winding temperature must derate conservatively — assume the worst case and give up performance the machine could physically deliver. An accurate estimate lets the controller run closer to the true thermal limit and use the motor’s short-term overload capability. The payoff is not ‘a nicer number’; it is usable torque.

7 Why is mean error the wrong headline metric here?

> Because this is a safety limit, not an average-case forecast. A model with 1 °C mean error and a 20 °C worst-case spike will still cook a motor. So alongside RMSE, MAE and R^2 I report **maximum absolute error** and **the fraction of samples within ±2 °C** — the two numbers a protection engineer would actually ask for.

8 Who is the stated customer, and what did they ask for?

> Problem Statement 6, GE Power Systems: improve operational reliability of electric motors by predicting winding temperature under diverse working conditions, using a supervised regression model on ambient temperature, motor speed, load torque, supply voltage and current. The virtual-sensor framing is my interpretation of *why* they would want it, and it is what makes the leakage decision in Section 4 obvious rather than arbitrary.

SECTION B — 8 QUESTIONS

Dataset

9 Describe your dataset.

> The *Electric Motor Temperature* dataset from Paderborn University's LEA department (`measures_v2.csv`, via Kaggle). Test-bench telemetry from a three-phase PMSM on a dynamometer: **1,330,816 samples** at **2 Hz** across **69 independent sessions**, about **185 hours** of running, 13 columns, **zero missing values**. The target spans 18.6–141.4 °C and sessions run 18–366 minutes.

10 What is `profile_id` and why does it matter so much?

> It labels the measurement session — one continuous recording of the bench under one excitation programme. It matters twice. First, every EWMA feature must be computed *within* a session, or the filter imports thermal state from an unrelated recording. Second, it is the grouping variable for the train/test split and for cross-validation. Essentially every correctness decision in this project is a `profile_id` decision.

11 What does the 2 Hz sampling rate imply?

> Samples are 0.5 s apart, and a motor's temperature cannot change appreciably in 0.5 s. So consecutive rows are near-duplicates. That single fact is why a random train/test split is invalid here, and it is also why subsampling the training set for the slower learners costs almost nothing — the data is heavily autocorrelated.

12 Which columns are inputs, and which are not?

> Eight inputs, all drive-measurable: `ambient`, `coolant`, `u_d`, `u_q`, `i_d`, `i_q`, `motor_speed`, `torque`. One target: `stator_winding`. Three columns dropped as leakage: `stator_tooth`, `stator_yoke`, `pm`. Plus `profile_id`, which is metadata, not a feature.

13 What is the dq frame, and why are voltage and current given as d and q?

> The dq (direct–quadrature) frame is a rotating reference frame locked to the rotor. Applying the Park transform to the three phase quantities turns sinusoidally-varying AC values into two slowly-varying DC-like components, which is what makes field-oriented control tractable: roughly, `i_q` produces torque and `i_d` controls flux. The dataset ships dq because that is what the controller works in.

14 Is dq convenient for *thermal* modelling too?

> No, and that is the point of Section 5.1. What heats a winding is copper loss, which depends on the *magnitude* of the current, not its decomposition. Handing a model `i_d` and `i_q` and expecting it to discover `i_s = sqrt(i_d² + i_q²)` and then square it is asking the model to rediscover physics I already know. I compute it instead.

15 Is the data clean? How do you know, rather than assume?

> I ran a systematic audit, not just a null check. Completeness: **0** NaN cells, **0** infinite cells, **0** duplicate rows, no constant columns. Structure: all **69** sessions are contiguous in file order, which is what makes the per-session EWMA safe. Plausibility: 0 rows above 200 °C, 0 above 6500 rpm, 0 above 400 A, 0 with ambient outside 0–50 °C. One check did fire, and I chased it down — see the next question.

16 Your audit flagged 69,386 rows (5.2 %) with winding temperature below ambient. Is that a sensor fault?

> No — it is a liquid-cooled machine, and that is the correct physical behaviour. In 99.9 % of those rows the *coolant* is also below ambient (median coolant 18.5 °C against median ambient 23.0 °C), and median current in those rows is 2.3 A against 110.6 A overall. So: a near-idle motor with chilled coolant running through it sits below room temperature. The flag was right to fire and the data is right to contain it. I kept the rows.

SECTION C — 4 QUESTIONS

Exploratory Data Analysis

17 What was the single most important thing EDA told you?

> That the problem is not instantaneous. Plotting excitation and thermal response on one time axis for the longest session shows current changing in *seconds* while winding temperature moves over *minutes*. The target at time t is not a function of the inputs at time t — it is a function of their recent history. Every design decision downstream follows from that one plot.

18 What is thermal inertia, in your own words?

> The motor has thermal mass. Electrical loss deposits energy into copper and iron, and that energy takes time to raise the temperature and time to escape to the coolant. The result is a low-pass relationship between loss and temperature, with time constants from tens of seconds (winding) through minutes (iron) to tens of minutes (housing and coolant).

19 What was the strongest raw correlation with the target, and what did you conclude?

> No raw electrical channel exceeded **$|r| = 0.63$**. The naive reading is 'weak signal, hard problem'. The correct reading is that the *instantaneous* relationship is weak because the physical relationship is not instantaneous. The ceiling is a property of the representation, not of the information content — and one transform lifted `i_s` from r ≈ 0.57 to r ≈ 0.70.

20 Why did you not use correlation to select features?

> Because correlation measures a linear, same-instant association, and the whole structure of this problem lives in the time domain. A correlation filter would have discarded exactly the channels that become predictive once you average them, and kept nothing useful. Feature *construction* was the task here, not feature selection.

SECTION D — 6 QUESTIONS

Pre-processing and Target Leakage

21 What is target leakage?

> When a feature carries information that would not be available at prediction time, usually because it is downstream of the target or is a proxy for it. The model scores brilliantly in validation and is useless in deployment, because the feature it leaned on does not exist in the real setting.

22 Which columns did you drop, and why?

> `stator_tooth`, `stator_yoke` and `pm`. Each correlates with the target above **r = 0.95**, but the reason for dropping them is not the correlation — it is that they are readings from *other thermocouples inside the same machine*. They are the very hardware the virtual sensor exists to replace. A model that uses them is not a virtual sensor; it is an interpolator between sensors you already said you cannot afford.

23 How did you know those three were leaking, rather than just useful?

> By asking the deployment question, not the statistical one: *would this signal exist on a shipped motor?* It would not, because installing those thermocouples is precisely the cost the project is avoiding. The r > 0.95 correlation is corroboration, not the argument. This is the single most consequential decision in the project — keeping them would have produced a far better number describing nothing.

24 Is dropping features correlated with the target not throwing away information?

> Yes, deliberately. The question is never 'does this feature help the score' but 'will this feature be present when the model runs'. Here the answer is no, so the information is not mine to use. Honest framing of the problem comes before accuracy.

25 Did you need to handle missing values, outliers or scaling?

> Missing values: none exist, so nothing to impute. Outliers: the audit found none that were physically implausible, and the one flag that fired was explained rather than removed — deleting real low-load data would have biased the model. Scaling: yes, but as a *pipeline step* rather than a pre-processing pass, so the scaler fits on training data only. See question 45.

26 Did you need to encode any categorical variables?

> No. Every input is continuous and physical. `profile_id` is categorical but is deliberately *not* a feature — it is grouping metadata. Encoding it would let the model memorise session identity, which is the opposite of what the split protocol is trying to test.

SECTION E — 12 QUESTIONS

Feature Engineering

27 How many features do you have, and where do they come from?

> 8 raw channels become **104 features**: the 8 raw inputs, plus **6** physically-derived instantaneous quantities, plus **90** thermal-history features — 9 channels × 2 statistics (exponentially weighted mean and standard deviation) × 5 time spans. 8 + 6 + 90 = 104.

28 What are the six derived physical quantities, and why those?

> `i_s = sqrt(i_d² + i_q²)` (current magnitude), `u_s = sqrt(u_d² + u_q²)` (voltage magnitude), `S_el = 1.5 · i_s · u_s` (apparent electrical power), `i_s²` (the copper-loss proxy), `P_mech = τ · n · 2π/60` (mechanical power), and `P_loss ≈ S_el − |P_mech|` (power that became heat). Each one is a term that appears in the heat balance of the machine.

29 Why `i_s²` specifically?

> Copper loss is `P = 3 I²R` — the heating is proportional to the *square* of the current. Giving the model `i_s²` hands it the correct functional form directly. A tree can approximate a square law with enough splits and a linear model cannot approximate it at all, so this is a free accuracy gain for anything linear and a free efficiency gain for anything else.

30 Derive the EWMA from the physics. This is the core of your project.

> A lumped-parameter thermal model of the motor is a first-order system: `C_th · dϑ/dt = P_loss(t) − (ϑ − ϑ_amb)/R_th`. Solving it gives `ϑ(t) = (1/τ) ∫ exp(−(t−s)/τ) P_loss(s) ds` with `τ = R_th · C_th`. That integral — an exponentially weighted average of past losses — **is** an EWMA. So computing EWMAs of the loss-related signals does not merely smooth the data: it supplies the exact functional form the physics produces. That is why even a linear model works once the features exist.

31 What is an EWMA, mathematically?

> A one-pole recursive low-pass filter: `y_k = α x_k + (1−&alpha) y_(k−1)`. Each output is a weighted average of all past inputs with weights decaying geometrically into the past. It needs one multiply–add and one stored value per sample, which is why it is as attractive for hardware as it is for the physics.

32 What is ‘span’, and how does it relate to α?

> pandas parameterises the filter by span *s*, with `α = 2/(s+1)`. I specify spans in *seconds* and convert to samples by multiplying by the 2 Hz sampling rate, so a 600 s span is 1200 samples. Working in seconds keeps the parameter physically meaningful — it is a thermal time constant, not a window length.

33 Why five spans, and why 30, 120, 600, 1800 and 3600 s?

> Because the machine is not one thermal mass but several, with different time constants: winding tens of seconds, iron minutes, housing and coolant tens of minutes. A single span would have to compromise. Five spans spanning 30 s to one hour bracket the physical range, and the model is left to weight them. The empirical answer agrees: the most-used features are the **600 s** ones, which sits right in the middle of that range.

34 Why also compute the exponentially weighted *standard deviation*?

> The mean tells you how much loss has recently been deposited; the standard deviation tells you how *volatile* the recent duty cycle has been. A motor held at a steady 100 A and one cycling violently around a 100 A mean have the same EWMA and different thermal states. The ewms terms earn their place — `coolant_ewms_120s` is the 5th most-used feature of all 104.

35 Why compute every EWMA within a `profile_id` group?

> Sessions are separate experiments, often with the machine cooled down in between. Letting a recursive filter run across a session boundary would carry the thermal state of an unrelated recording into the first minutes of the next one. That is leakage, and it is silent — it would quietly improve the score while corrupting the physics. I use `groupby("profile_id").transform(...)` so each session's filter starts fresh.

36 Why `adjust=False`?

> `adjust=False` gives the true recursive form `y_k = αx_k + (1−α)y_(k−1)`, which is exactly what a hardware implementation would compute and exactly what the thermal ODE solution looks like. `adjust=True` applies a finite-sample renormalisation that is statistically tidier but no longer the physical filter, and could not be reproduced on a controller with one register per channel.

37 Did the instantaneous physical features actually help? Be honest.

> Measured, with an ablation on identical LightGBM models. Raw inputs only: RMSE **14.39 °C**, R^2 0.6942. Adding the six instantaneous derivations: **14.58 °C**, R^2 0.6860 — very slightly *worse*, inside noise. Adding the EWMAs: **2.43 °C**, R^2 0.9913. So the instantaneous derivations buy essentially nothing on their own; the entire gain is thermal history. I keep them because they are the *channels* the useful EWMAs are computed *of*, and because they help the linear models.

38 Which features does the model actually rely on?

> All of the top 15 by LightGBM split count are EWMA-derived. The top three are `coolant_ewma_600s` (2234 splits), `i_s_sq_ewma_600s` (1935) and `i_s_ewma_600s` (1841). Read that physically: the 600 s average of coolant temperature is the **heat-sink boundary condition**, and the 600 s average of `i_s²` is the **copper-loss term**. Those are precisely the two terms the lumped thermal ODE names as dominant. The model learned the physics the features were designed around, which is the strongest evidence available that the result is not an artefact.

SECTION F — 6 QUESTIONS

Train / Test Split Protocol

39 How did you split the data, and why that way?

> Profile-wise: entire measurement sessions are held out. Test is profiles **65 and 72** (**55,395** samples), validation is profiles 60, 62 and 74 (**63,904** samples), training is the remaining **64** profiles (**1,211,517** samples). The model is evaluated the way it would be used: on a motor run it has never seen.

40 What exactly goes wrong with `train_test_split(X, y, test_size=0.2)` here?

> Rows are 0.5 s apart and near-identical. A random split scatters near-duplicate rows across both sides, so the test set is effectively already inside the training set. The model is rewarded for memorising rather than generalising, and the reported score describes nothing you could deploy.

41 Quantify it. How wrong is a random split?

> Same model, same features, only the split changes. Random 80/20 over rows: RMSE **0.66 °C**, R^2 **0.9995**. Profile-wise: RMSE **2.50 °C**, R^2 **0.9908**. Random splitting understates error by **3.8×**. I ran this deliberately, as a study, because the 0.9995 is the number most projects on this dataset report.

42 How did you choose profiles 65 and 72 specifically?

> They are the benchmark test sessions used in the published literature on this bench, so my numbers are comparable to other people's. Choosing them in advance also removes any suspicion that the test set was selected to flatter the result.

43 What did the validation set do, if the test set is what you report?

> Early stopping and model selection. LightGBM's early stopping watched profiles 60, 62 and 74 and settled at **743** trees out of a 900 budget; the MLP used scikit-learn's internal early stopping. The test profiles were touched exactly once, at the end. Using the test set to choose a stopping point is a subtler form of the same leakage the split protocol exists to prevent.

44 You subsampled the training set for some models. Does that not throw away data?

> For the Random Forest and MLP I trained on every 10th row — **121,152** samples. At 2 Hz, consecutive rows are near-duplicates, so the information lost is small while the training time saved is large. Crucially, subsampling is applied *within* the training set only; the test profiles are evaluated at full rate, and the EWMA features were computed at full rate before any subsampling, so no thermal history is lost.

SECTION G — 9 QUESTIONS

Model Development

45 Which models did you try, and why that ladder?

> Five, deliberately ordered from the simplest thing that could work upward: linear regression on raw inputs, Ridge on the full feature set, Random Forest (80 trees, depth 16), an MLP (two hidden layers of 128), and LightGBM (900-tree budget, early-stopped to 743). The point of a ladder is the *comparison* — it isolates how much each addition is worth. The first two rungs alone carry the project's main finding.

46 What is the main finding from the ladder?

> Linear regression on raw inputs: RMSE **17.79 °C**, R^2 **0.5324**. The same algorithm on the engineered features: RMSE **2.08 °C**, R^2 **0.9936**. No change of model — only features. **Feature engineering beat model sophistication**, and by a margin no amount of tuning could have recovered. That is the sentence I would want remembered.

47 Why Ridge rather than plain least squares on 104 features?

> The 104 features are heavily collinear by construction — a 30 s and a 120 s EWMA of the same channel are nearly the same signal. Ordinary least squares on collinear inputs gives unstable, wildly-scaled coefficients. The L2 penalty keeps the solution well-conditioned without discarding any feature, which suits a feature set whose redundancy is intentional.

48 Your final model is an MLP. Why did it beat both tree ensembles?

> Extrapolation. Tree ensembles are **piecewise-constant**: a prediction is the mean of the training samples in a leaf, so a tree can never output a value outside the range it was trained on. The held-out sessions contain operating points the 64 training profiles never visit, and at those points a tree can only clamp to its nearest leaf. An MLP is a smooth function and continues the trend. Under genuine extrapolation, smoothness is the decisive property — which is why the usual 'trees-win-on-tabular-data' expectation fails here.

49 Can you prove the test set actually requires extrapolation, rather than asserting it?

> Yes. Marginally, every feature's range overlaps between train and test, so a per-feature check says nothing. The question is joint. Measuring nearest-neighbour distances in the standardised 104-dimensional feature space, the median distance from a test point to its nearest training point is about **5.6×** the median distance between training points themselves, and roughly **9 %** of test points lie beyond the 99th percentile of intra-training distances. The exact ratio depends on how densely you sample the training reference set, but the direction is robust: the held-out sessions sit in a part of joint feature space the training data does not populate.

50 Describe your final model precisely.

> A scikit-learn `Pipeline` of `StandardScaler` followed by `MLPRegressor(hidden_layer_sizes=(128, 128), alpha=0.01, learning_rate_init=0.001, early_stopping=True, n_iter_no_change=6, max_iter=80, random_state=42)`. ReLU activations, Adam optimiser, 104 inputs, one output. Serialised with the feature list, the target name, the test metrics and the scikit-learn version into a **344 KB** joblib bundle.

51 Why `StandardScaler`, and why inside a `Pipeline` rather than before the split?

> The MLP needs it: features span volts, amperes, rpm, watts and degrees Celsius, and gradient descent on unscaled inputs of wildly different magnitudes converges badly. Putting it *inside* the pipeline means the scaler's mean and variance are fitted on training folds only. Scaling the whole dataset first would let test-set statistics influence the transform — a textbook, easily-missed leak, and one that cross-validation would not catch because every fold would already be contaminated.

52 Why did you not use an LSTM or a temporal CNN, given this is time-series data?

> Two reasons, one principled and one practical. Principled: the physics tells me the exact functional form — an exponentially weighted integral of past losses — so I can *construct* that representation rather than asking a network to learn it from data. That is cheaper, interpretable, and it worked. Practical: the resulting model is 90 registers and ~30k multiply-accumulates, which runs on a motor-control MCU; a sequence model does not. I list sequence models in Section 12 as the natural next step, specifically to learn the time constants instead of fixing five spans.

53 Why `random_state=42` everywhere?

> Reproducibility. The MLP's weight initialisation, the Random Forest's bootstrap, LightGBM's subsampling and the randomised search's candidate draws are all stochastic. Fixing the seed means the reported numbers can be regenerated exactly. Small differences across machines are still possible from BLAS threading in the MLP, and I say so rather than claiming bit-exactness.

SECTION H — 9 QUESTIONS

Performance Evaluation and Interpretation

54 Define your four regression metrics.

RMSE = sqrt(mean((y − ŷ)²)) — typical error in °C, penalising large misses quadratically. **MAE** = mean(|y − ŷ|) — typical error, all misses weighted equally. **R²** = 1 − SS_res/SS_tot — the fraction of variance explained, dimensionless. **MaxAE** = max(|y − ŷ|) — the single worst prediction in the test set.

55 Why report ‘within ±2 °C’ at all?

Because it is the question an engineer actually asks: *how often is this estimate good enough to act on?* RMSE is a single summary statistic that can hide a bimodal error distribution; a coverage fraction cannot. For the final model, **84.2 %** of samples fall within ±2 °C.

56 State your final results.

Tuned MLP, on profiles 65 and 72 — 55,395 samples, 7.7 hours, never seen in training: RMSE **1.54 °C**, MAE **1.10 °C**, R² **0.9965**, max absolute error **11.32 °C**, **84.2 %** within ±2 °C.

57 Walk through the model-comparison table.

Linear raw 17.79 °C / R² 0.5324. Ridge + EWMA 2.08 / 0.9936. Random Forest 3.60 / 0.9808. MLP untuned 1.67 / 0.9959. LightGBM 2.50 / 0.9908. Tuned MLP **1.54 / 0.9965**. Tuned HistGradientBoosting 2.39 / 0.9915. Three readings: features matter more than the model; the neural net beats the ensembles because of extrapolation; and **RMSE and max error do not rank the models identically** — tuned HGB has the best worst case at **9.53 °C** despite worse RMSE.

58 So which model would you actually deploy?

It depends on the selection criterion, and I would say so explicitly rather than pretend there is one answer. I selected on RMSE, which gives the MLP. If worst-case error were the criterion — and for a protection function it arguably should be — tuned HistGradientBoosting at **9.53 °C** MaxAE is the better choice, at the cost of RMSE rising from 1.54 to 2.39. Selecting on RMSE and then disclosing max error is the honest order of operations.

59 Where does the remaining error actually live? Average error hides structure.

I broke it down three ways. By operating temperature: median error stays near 1 °C in every band from 18 °C to 145 °C — accuracy does not decay when the motor is hot, which is the property that matters since a thermal limit is only consulted in the hot regime. By load: likewise flat, with a moderately wider tail under heavy load. By time into the session: **this is where the structure is** — the largest errors are concentrated in the first minutes, while the 1800 s and 3600 s filters are still converging from their zero initial state.

60 Is the model biased?

> The residual distribution is approximately zero-mean and symmetric, and residuals plotted against measured temperature show no systematic trend — so no consistent over- or under-prediction across the operating range. The error is dispersion, not bias. For a thermal limit that is the preferable failure mode, but it is also why the max-error number still has to be reported separately.

61 R^2 = 0.9965 sounds too good. Why should I believe it?

> You should be suspicious, and I built two studies specifically to answer that. First, the leakage study: I show what the inflated number looks like (R^2 0.9995 under a random split) and report the honest one instead. Second, the ablation: the accuracy is traceable to a specific, physically-justified feature family rather than appearing from nowhere. And the headline R^2 is computed on two entire sessions that no training sample came from. The high R^2 is partly just the target's wide variance — 18 to 141 °C — which is why I lead with RMSE in °C, not R^2.

62 What does the predicted-versus-measured plot show?

> The two held-out sessions overlaid as time series, prediction against measurement. The estimate tracks every ramp, plateau and cool-down, with visible divergence only in the opening minutes of profile 72 — the warm-up effect. It is the most informative figure in the project because it shows error *in time*, which a scatter plot or a single metric cannot.

SECTION I — 7 QUESTIONS

Hyperparameter Tuning and Cross-Validation

63 How did you tune, and what did you tune?

> `RandomizedSearchCV` over the two strongest families. For the MLP: hidden layer sizes, L2 penalty `alpha`, and initial learning rate — 8 candidates. For HistGradientBoosting: `max_iter`, learning rate, `max_leaf_nodes`, `min_samples_leaf` and L2 regularisation — 10 candidates. Three grouped folds each, scored on negative RMSE, on a 1-in-10 subsample of the non-test data.

64 Why randomised search rather than an exhaustive grid?

> The MLP grid alone is 4×4×3 = 48 combinations and each fit takes minutes; the full grid across both families is hundreds of fits. Randomised search samples the space, and with most hyperparameters mattering far less than a few, random sampling finds a near-optimal configuration for a fraction of the compute. The MLP search took **6.9** minutes and the HGB search **11.8**.

65 Why `GroupKFold` and not plain `KFold`?

> Because a plain `KFold` would split *within* sessions and silently reintroduce exactly the leakage the whole split protocol exists to prevent — near-duplicate rows on both sides of each fold boundary. Every candidate would be scored on a contaminated fold, so the search would rank them by how well they memorise. `GroupKFold` grouped on `profile_id` keeps whole sessions together. This is not optional here.

66 Your CV RMSE is about 3 °C but your test RMSE is 1.54 °C. That looks wrong. Explain.

> It is expected, and it is a point worth making. Each grouped fold trains on only about two-thirds of the available sessions, so every candidate sees far less thermal diversity than the final model does when it is refitted on all 64 training profiles. Grouped CV here is for **ranking candidates**, not for estimating deployed accuracy. The MLP scored CV RMSE **3.04** against HGB's **4.02**, and that ordering held on the test set — which is all I asked of it.

67 What were the winning hyperparameters?

> MLP: `hidden_layer_sizes=(128, 128)`, `alpha=0.01`, `learning_rate_init=0.001`. Notably the search chose the *same* architecture as my initial hand-picked guess and changed only the regularisation and learning rate — so most of the gain came from `alpha` rising to 0.01, i.e. more regularisation, which is consistent with the extrapolation story.

68 Did tuning help?

> Partly, and I report both halves. RMSE improved **8 %** (1.67 → 1.54 °C) and MAE **13 %** (1.26 → 1.10 °C). But maximum absolute error got *worse*, 9.84 → **11.32 °C**, because the search objective was RMSE and nothing in it constrained the tail. That is a real cost for a safety-limit application, and hiding it would have been the easy thing to do.

69 Why `clone(rs.best_estimator_)` before the final refit?

> Because `best_estimator_.__class__(**get_params())` does not reconstruct a `Pipeline` correctly — a pipeline's parameters include its nested steps, and naively reinstantiating it loses them. `clone` produces an unfitted copy with identical hyperparameters, which is exactly what you want before refitting on the full training set.

SECTION J — 10 QUESTIONS

Deployment, Limitations and Extensions

70 Could this actually run on a real motor controller?

> Yes, and that is what makes it a thermocouple replacement rather than an offline analysis. The MLP is two layers of 128 over ~104 inputs — roughly **30,000 multiply-accumulates** per inference, at the 2 Hz rate the thermal model needs. That is negligible for any modern motor-control MCU and trivial on an FPGA.

71 What does the feature extraction cost in hardware?

> Almost nothing, which is the elegant part. Each EWMA is `y_k = αx_k + (1−α)y_(k−1)` — one multiply-add and one register. All **90** EWMA features cost 90 registers and 90 MACs per sample. A hardware-friendly feature extractor is unusual, and it is a large part of why this formulation is attractive for deployment rather than just for a report.

72 What numeric precision would you need?

> Temperatures span 18–141 °C and need about 0.1 °C resolution, so 16-bit fixed point suffices at the output. The EWMA accumulators need more headroom because of the long time constants — a 3600 s span means `α ≈ 2.8×10⁻⁴`, so small increments accumulate over thousands of samples. Q16.16 accumulators are a safe starting point.

73 What is the warm-up problem, and how would you fix it?

> The EWMA states initialise at zero, so for the first minutes of a session the 1800 s and 3600 s features are only partly converged and the predictions built on them are correspondingly worse. It is the largest single remaining error source and it is intrinsic to a recursive feature set. The cheapest fix is to seed each filter from the first sample instead of zero, which costs nothing and removes most of the transient; that is first on my improvement list.

74 What are the real limitations of this work?

> Eight, and I would rather state them than be asked. Worst-case error is ~11 °C, so a deployed limit still needs a safety margin. One machine, one test bench — nothing proves transfer to other motor geometries. Warm-up, as above. Tuning improved the average but not the tail. Five training sessions (profiles 47, 46, 52, 64, 51) are shorter than the 3600 s span, so ~1.5 % of training rows carry a partly-formed feature — none are in validation or test, so the results stand, but strictly not every feature means the same thing in every row. The CV-to-test gap is large. Session identity is required at inference. And there is no uncertainty estimate.

75 Why does ‘session identity is required at inference’ matter?

> Because the model is not a pure function of an instantaneous sample — it needs correctly segmented, chronologically ordered history, since the features are averages over that history. In deployment that means the drive has to know when a run begins so it can reset the filters, and it cannot feed samples out of order. That is a real integration requirement, not a detail.

76 A protection function wants a bound, not a point estimate. What would you do?

> Quantile regression or conformal prediction, to produce a defensible upper bound rather than a mean estimate. None of my five models provides one, and for a safety limit that is arguably a more valuable improvement than shaving RMSE. It is on the improvement list for exactly that reason.

77 What would you do next, in priority order?

> Seed the EWMA states from the first sample — cheapest fix for the largest remaining error. A physics-informed loss enforcing the thermal ODE, which would constrain trajectories to physically realisable ones and should attack the worst case specifically. Quantile or conformal prediction for an upper bound. A sequence model to learn the time constants instead of fixing five spans. Multi-output prediction of all four temperatures, to exploit the shared thermal structure between winding, tooth, yoke and magnet.

78 If I only remember one thing from your project, what should it be?

> That the result came from understanding the physics, not from trying models. The same linear regression goes from R^2 0.53 to 0.99 with no change of algorithm, because the features now encode the exponentially weighted integral that the thermal ODE actually produces. And the second thing: the evaluation protocol changes the headline by 3.8×, so a number without its protocol is not a result.

79 What was the hardest decision you made?

> Dropping `stator_tooth`, `stator_yoke` and `pm`. They correlate above 0.95 with the target and keeping them would have produced a far prettier number. Dropping them is what makes the project a virtual sensor instead of a curve fit, and it cost most of the accuracy I then had to earn back through feature engineering.